

STANFORD UNIVERSITY RESEARCH

Data-Efficient Pre-training by Scaling Synthetic Megadocs

Konwoo Kim, Suhas Kotha, Yejin Choi,
Tatsunori Hashimoto, Nick Haber, Percy Liang



arXiv:2603.18534 · March 2026

Research Journey

01

Introduction & Problem Statement

The growing gap between compute and data availability in language model pre-training

02

Methodology Overview

Synthetic data generation techniques and evaluation framework

03

Simple Rephrasing Results

Scaling synthetic generations and loss improvements

04

Megadoc Methods

Stitched rephrasing and latent thoughts for better scaling

05

Scaling Analysis

Data efficiency metrics and performance comparisons

06

Ensembling & Composition

How synthetic data benefits compose with model ensembling



Key Innovation: Improving data efficiency from 1.48× to 1.80× through megadoc techniques



CHAPTER 01

Introduction & Problem Statement

The growing gap between compute and data availability in language model pre-training



THE FUNDAMENTAL PROBLEM

The Data-Compute Challenge



Compute Growth

Training compute for frontier AI models grows by 4-5× per year, following exponential scaling trends observed over the past decade.

Source: Sevilla & Roldán (2024), Epoch AI



Data Limitation

Web text data is fundamentally limited - high-quality human-generated text can not scale at the same rate as compute.

Source: Villalobos et al. (2024)

The Critical Gap

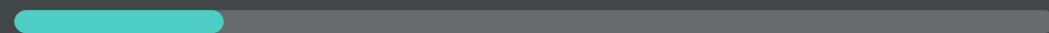
Compute Growth Rate

4-5× / year



Data Growth Rate

-1× / year



This divergence creates an urgent need for data-efficient pre-training methods



Why It Matters

Without data-efficient methods, we cannot fully utilize available compute, leading to suboptimal model performance and wasted resources.

Research Objectives & Approach

Core Research Questions

- 1 Can synthetic data improve i.i.d. validation loss on web text despite coming from a different distribution?
- 2 How can we design algorithms that achieve better loss scaling as compute approaches infinity?

Key Innovation

We operate under the **asymptotically infinite compute regime**, training the best possible model given fixed data under unlimited training compute.

Following Kim et al. (2025) framework

Experimental Setup

 Model Size

300M

parameters

 Real Data

200M

tokens (DCLM)

 Training

Unlimited

compute regime

 Generator

Llama 3.1 8B

Instruct

Why External Generator?

- ✓ Models at our scale cannot reliably follow instructions
- ✓ Pre-training generator from scratch too expensive
- ✓ Ablations suggest improvements hold for self-trained generators

 **Goal:** Model the original web distribution, not just transform data

Synthetic Data Generated by AI

CHAPTER 02

Methodology Overview

Synthetic data generation techniques and evaluation framework



Production Data

Create Ruleset

Generate Test Data

Preparation
by Domain Expert

Data Generation
by LLM

Synthetic
Test Dataset



- Data Privacy Concerns
- High Hallucination Rates
- Limited Data Variety
- No Referential Integrity
- No Data Consistency
- No Enterprise Scalability
 - DevOps Integration
 - Centralized Mgt.
 - Analytics & Reporting
 - Global Self-Service

Synthetic Data Generation Pipeline

1 Simple Rephrasing

Process

Query generator to rephrase each real document into English Wikipedia article format

Parameters

- Temperature: 1.0
- Max length: 1024 tokens
- Avg output: 708 tokens

Output

G independent rephrases per document, shuffled as i.i.d. samples

2 Stitched Rephrasing

Process

Concatenate all G rephrases + real doc from same source into single megadoc

Variants

- Real First: Doc → Rephrases
- Real Last: Rephrases → Doc (better)

Key Insight

Preserves document relationships; megadocs often exceed 4096 context window

3 Latent Thoughts

Process

Split doc into G+1 chunks, generate rationales connecting prefix → suffix

Format

Insert rationale between chunks; avg 424 tokens per thought

Advantage

Directly stretches document with reasoning traces; better token efficiency



All methods use cross-document attention (no masking) to enable megadoc learning



Evaluation Metrics & Baseline

Primary Metrics



i.i.d. Loss

Validation loss on original web distribution **without synthetic data mixing**



Data Efficiency

How much **less real data needed** to achieve same loss using scaling laws



Long-Context Loss

Loss on **arXiv CS papers** (naturally long documents)



Benchmark Accuracy

PIQA, SciQ, ARC Easy – high-signal benchmarks at our scale

Hyperparameter Search



Learning Rate

{0.001, 0.003, 0.01}



Weight Decay

{0.1, 0.4, 0.8, 1.6}



Epoch Count

{1, 2, 4, 8, 16, 32}



Mixing Fraction

{0.5, 0.75, 0.9}



Locally Optimal: Deviation along any dimension doesn't improve loss

Baseline Performance

Model

300M Parameters

Data

200M Real Tokens

Training

Optimal Epoching + Regularization

Baseline i.i.d. Loss

3.55

Why These Benchmarks?

Following Thrush et al. (2025), PIQA, SciQ, and ARC Easy are **high-signal benchmarks** at our pre-training scale, providing reliable evaluation of model capabilities.

CHAPTER 03

Simple Rephrasing Results

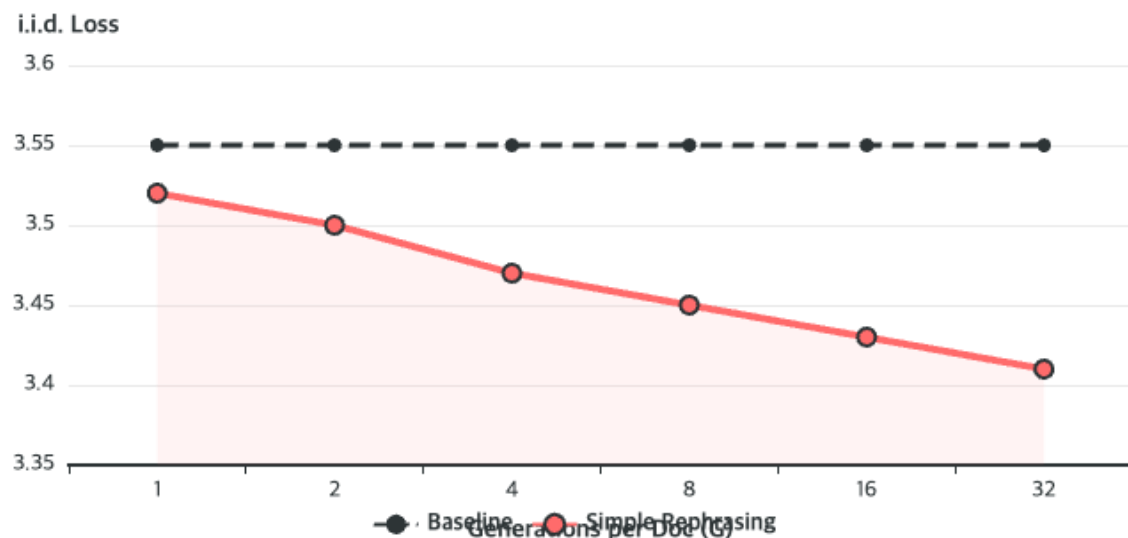
Scaling synthetic generations and loss improvements



KEY FINDING

Loss Scaling with Rephrasing

i.i.d. Loss vs. Generation Count



Key Observations

- ✓ **Monotonic improvement:** Loss decreases as G increases
- ✓ **Plateauing behavior:** Appears to saturate around 32 generations
- ✓ **No overfitting:** Improvements hold with careful tuning
- ✓ **Distribution mismatch:** Works despite synthetic data being different

Downstream Impact

PIQA +5%

SciQ +5%

ARC Easy +5%

Average accuracy improvement across benchmarks

Baseline Loss

3.55

Real data only

Best Loss @ 32G

3.41

-0.14 improvement

Data Efficiency

1.48×

48% less data needed

CRITICAL SUCCESS FACTOR

Hyperparameter Optimization

Why Tuning Matters

Careful hyperparameter search enables **monotonic loss improvement** – larger G can always use same hyperparameters as smaller G while epoching synthetic stream less.



Without tuning: Confuses introducing synthetic data with epoching real data

Locally Optimal Search

1 Seed Search

Start with initial runs around best-guess optimal hyperparameters

2 Iterative Refinement

Take best run, evaluate all neighbors, repeat if improvement found

3 Certification

Terminate when no neighbor improves loss – locally optimal

Optimal Hyperparameters @ 16 Generations

Learning Rate

0.003

Weight Decay

0.8

Real Epochs

16

Mixing Fraction

0.75

Mixing Strategy

Real Stream

D real documents with tuned epoching

Synthetic Stream

G × D rephrases + D real docs, epoched indefinitely

Concat-and-Chunk

Select stream, fill context window with random permutation

CHAPTER 04

Megadoc Methods

Stitched rephrasing and latent thoughts for better scaling



Stitched Rephrasing: Concept & Benefits

Core Innovation

Traditional Approach

Shuffle all G rephrases as **independent i.i.d. documents**, losing document relationships

Stitched Approach

Concatenate all G rephrases + real doc into **single megadoc**, preserving relationships

Inspired by: In-context Pre-training (ICPT) [Shi et al., 2024] – but stitching is **essentially free** since we know a priori which generations are related

Two Variants

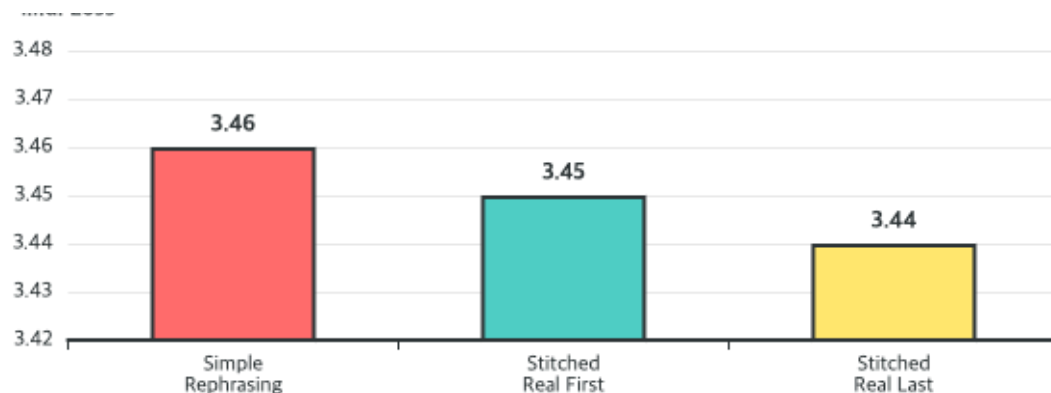
A Real First

Real Doc $\rightarrow$ Rephrase 1 $\rightarrow$... $\rightarrow$ Rephrase G

B Real Last (Better)

Rephrase 1 $\rightarrow$... $\rightarrow$ Rephrase G $\rightarrow$ Real Doc

Performance Results @ 8 Generations



Why Real Last Wins

May relate to **epiplexity** [Finzi et al., 2026]: learning to generate real doc from rephrase is more valuable than the reverse.

Hypothesis: The inverse task (rephrase $\rightarrow$ real) imparts more generalizable structure since rephrases are less detailed.

📈 Long-context loss shows even larger improvements

Latent Thoughts: Rationale-Based Augmentation

Process Overview

1

Split Document

Divide into G+1 equal chunks

2

Generate Rationales

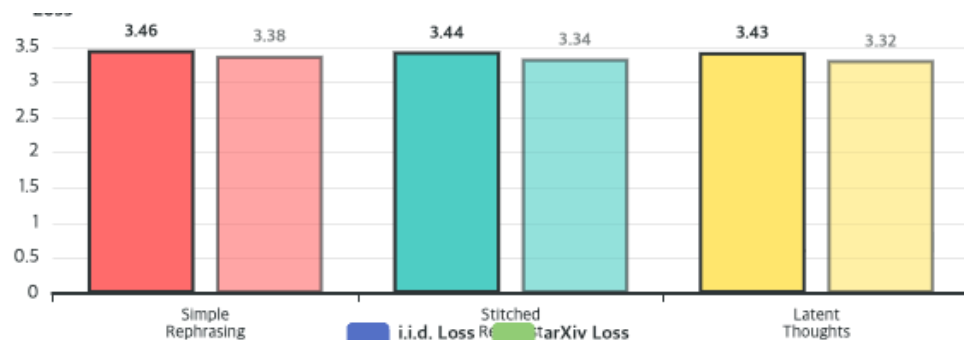
For each prefix→suffix pair

3

Stretch Document

Insert tags

Performance Comparison @ 8 Generations



Key Characteristics

📄 Avg Thought Length

424 tokens

✂ More Token-Efficient

Shorter than rephrasing (708 tokens)

🧠 Reasoning Traces

Captures implicit knowledge & logic

Prompt Design

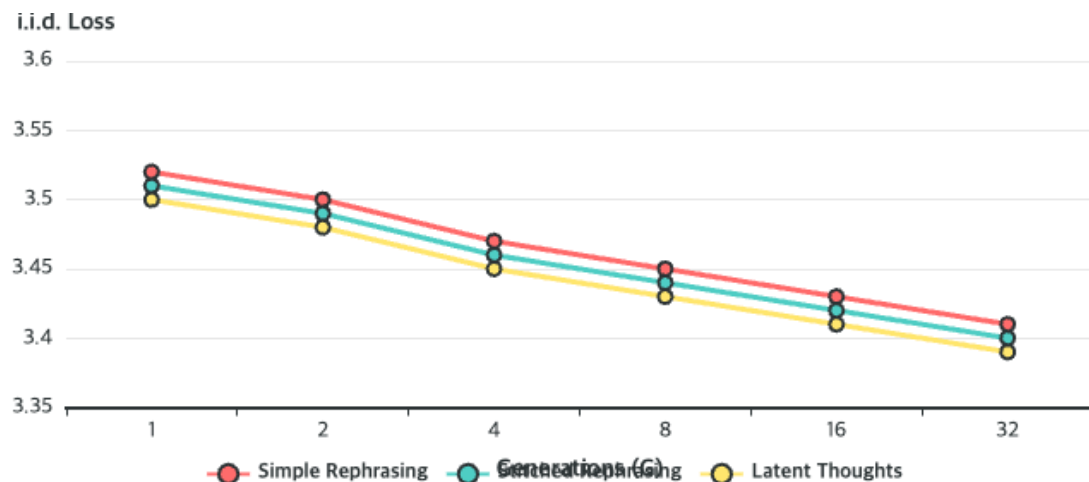
[Rationale connecting prefix to suffix]

- ✓ Concise, declarative language
- ✓ Focus on suffix, avoid repetition
- ✓ Concrete reasoning & derivations

🏆 Best overall performer across all metrics

Megadoc Scaling Analysis

Data Efficiency @ 32 Generations



Key Scaling Insights



Less Plateauing

Megadocs show fewer signs of saturation as G increases



Widening Gap

Improvement grows: 0.02 @ 4G → 0.07 @ 32G



Better Token Efficiency

Latent thoughts shorter than rephrases

Long-Context Results @ 32G

Stitched

vs. simple rephrasing

-0.14

Latent Thoughts

vs. simple rephrasing

-0.19

Measured on arXiv CS papers

Simple Rephrasing

1.48×

Baseline improvement

Stitched

1.64×

+11% better

Latent Thoughts

1.80×

+22% better



Best: 9% benchmark accuracy improvement

Why Megadocs Scale Better

Dual Benefit Hypothesis

1 Constant Improvement

Megadoc structure provides **immediate loss reduction** independent of training length

2 Better Scaling

Megadocs **enable longer training** without overfitting, extracting more value from limited data

Evidence: Longer Training Enabled

Real Data Epochs

Simple	16
Megadocs	32

Mixing Fraction

Simple	0.75
Megadocs	0.90

🔑 Result: 5× more training steps without overfitting

Controlled Experiment

When using **simple rephrasing hyperparameters** for megadoc methods, the favorable scaling disappears.

🔧 Megadocs still provide **constant improvement**, but scaling advantage is lost

Short vs. Long Context

Surprisingly, megadocs improve **both short and long documents**:

Short Docs (<600 tokens)

Median 304 tokens, max 621

Better

Long Docs (>600 tokens)

Median 1230 tokens, max 36,319

Better

💡 Megadocs provide better learning task with general transfer

CHAPTER 05

Ensembling & Composition

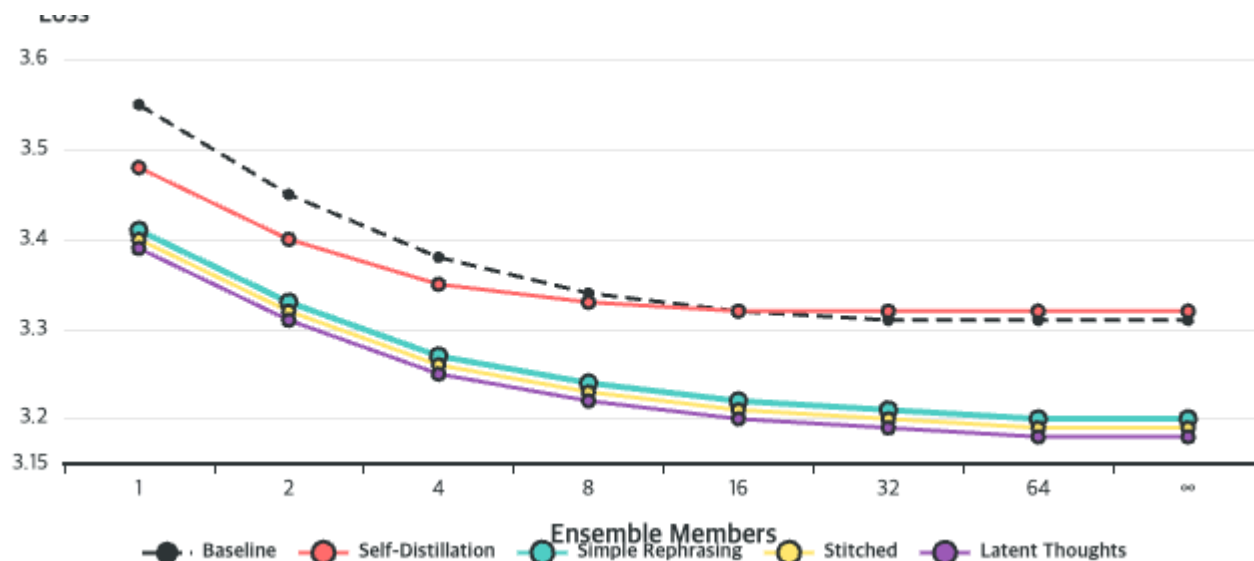
How synthetic data benefits compose with model ensembling



UNIQUE COMPOSITION

Ensembling Results: Unique Composition

Ensemble Loss vs. Member Count



Key Finding

Not all synthetic data composes with ensembling.

- ✗ Self-distillation: No asymptote improvement
- ✓ Rephrasing: Lowers asymptote by ≥ 0.12

Why It Matters

This demonstrates a **qualitatively unique benefit**:

- ★ Synthetic data augmentation $\neq$ Ensembling
- ★ Synthetic data augmentation $\neq$ Self-distillation
- ★ Benefits are complementary and stack

Self-Distillation

3.32

Same as baseline (3.31)

✗ Does NOT compose

Synthetic Data

3.20

-0.12 improvement

✓ DOES compose

💡 All three methods compose with ensembling

Student Scaling & Self-Improvement

Student Scaling Ablations

Concern: Does synthetic data only help because student is weaker than generator?

 **Test: Scale student 5× (300M → 1.5B) with same generator**

300M Student

+0.14-0.21

Loss improvement

1.5B Student

+0.21-0.27

Loss improvement

↑ Stronger students benefit MORE from synthetic data!

Prior Work: Self-Improvement

Kotha et al. (2025)

Llama 3.1 8B improved itself by **16.6%** on factuality task via self-rephrasing

Yang et al. (2025)

Modeling inter-document relationships improved models trained on same data

Ruan et al. (2025)

On-policy latent thoughts provide useful training signal

Three Reasons for Optimism

1 Stronger Students Benefit More

Ablations show larger loss wins at 1.5B scale than 300M scale for all methods

2 Self-Improvement Demonstrated

Prior work shows rephrasing, inter-document modeling, and latent thoughts enable self-improvement

3 Vision Precedent

Simple augmentations (blur, rotate) help at all scales in vision – rephrasing may operate similarly

Generator Scaling Observations

- ☒ Weak generators (1B) don't produce meaningful synthetic data
- ☒ Scaling generator past 8B provides diminishing returns
- ☒ Llama 3.1 8B likely near upper limit of generator scaling

 Gap between student & teacher isn't the key ingredient

Key Takeaways & Future Directions

Key Findings

- ✓ **Synthetic data improves i.i.d. loss** on web text despite distribution mismatch
- ✓ **Megadocs boost data efficiency** from 1.48× to 1.80× at 32 generations
- ✓ **Benefits compose uniquely** with ensembling, distinct from self-distillation
- ✓ **Stronger students benefit more** from synthetic data augmentation

Future Work

- 1 Test findings at scale with **self-trained generators** from scratch
- 2 Explore **hierarchical prompting strategies** for megadocs
- 3 Extend to **other domains** (vision, multimodal) and architectures
- 4 Investigate **optimal megadoc length** and structure



Open Source

All runs available on WandB · Code on GitHub

